Production-Grade End-to-End Data Pipeline

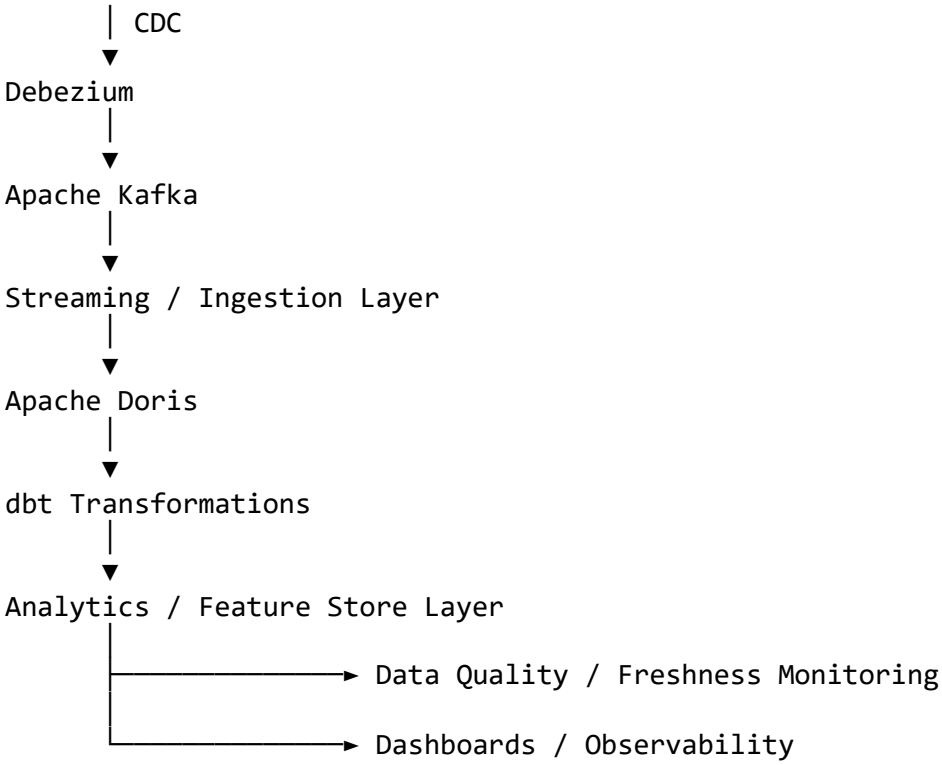
Table of Contents

1. Overview

This project implements a production-oriented data engineering pipeline designed to demonstrate reliable ingestion, CDC processing, transformation, orchestration, data quality validation, monitoring, and CI/CD.

The pipeline follows an end-to-end flow:

```
Source Database
      |
```



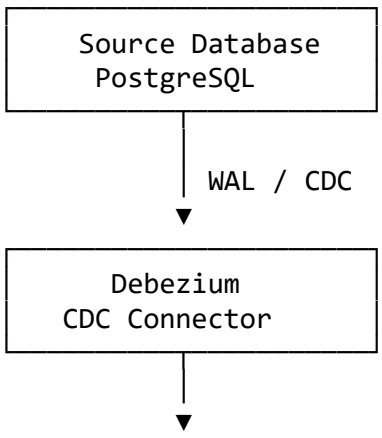
Apache Airflow is used to orchestrate scheduled workflows and provide operational visibility into pipeline execution.

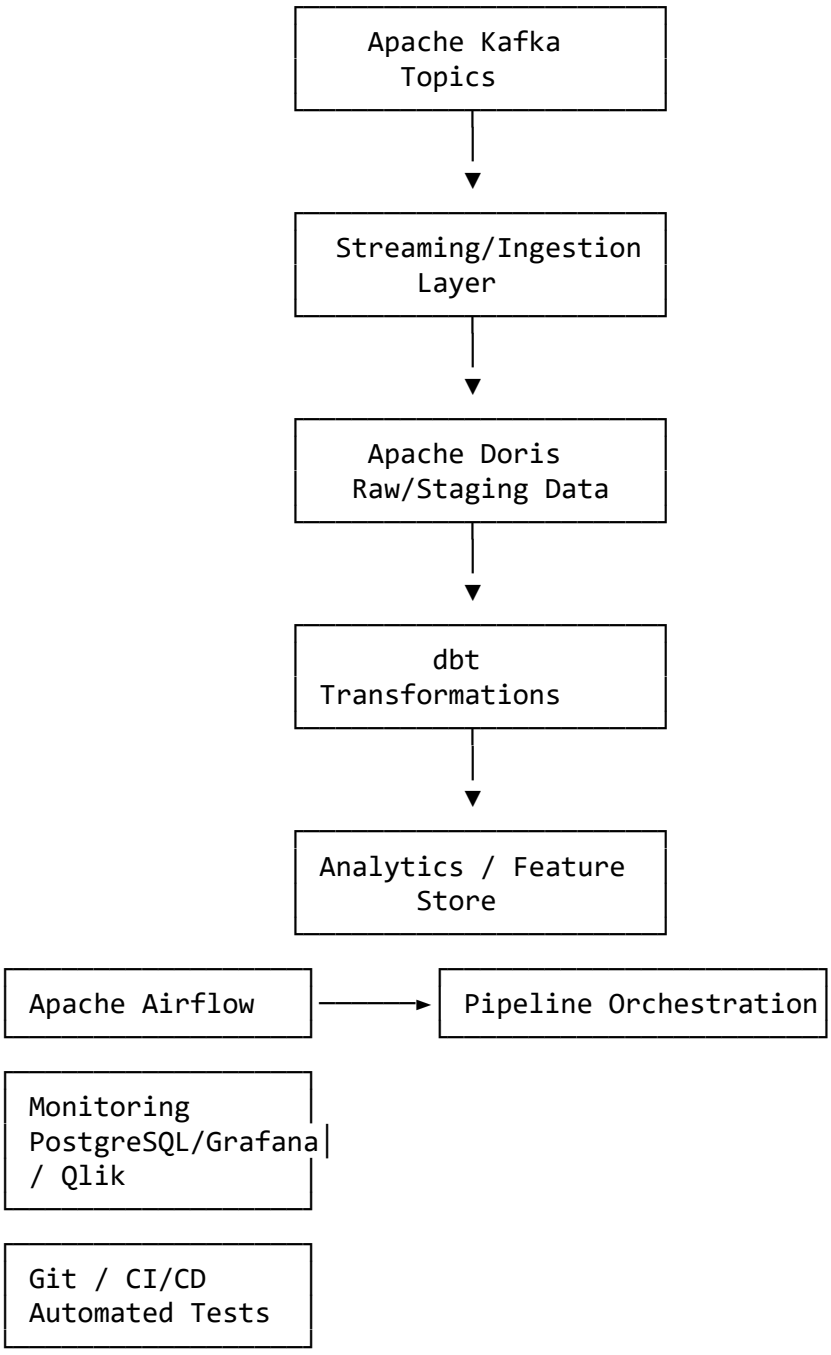
The implementation also includes monitoring for: pipeline execution, source-to-target data movement, table freshness, record counts, ingestion lag, routine load / streaming ingestion status, data quality, pipeline failures, and CDC connector health.

2. Architecture

The solution is based on a layered architecture separating ingestion, storage, transformation, orchestration, and observability.

High-level architecture





Pipeline layers

Layer	Technology	Responsibility
Source	PostgreSQL / source systems	Operational data
CDC	Debezium	Capture inserts, updates and deletes
Messaging	Apache Kafka	Reliable event transport
Ingestion	Kafka/Doris streaming ingestion	Move CDC events into analytical storage

Storage	Apache Doris	Analytical storage
Transformation	dbt	SQL transformations and data modelling
Orchestration	Apache Airflow	Scheduling and dependency management
Monitoring	PostgreSQL / Grafana / Qlik	Pipeline and data observability
CI/CD	Git-based CI/CD	Automated validation and deployment

3. Data Flow Explanation

3.1 Source Database

The pipeline starts from the operational source database. The source contains transactional tables whose changes need to be propagated downstream without repeatedly extracting the entire table. CDC is therefore used instead of performing full-table extracts.

3.2 Change Data Capture

Debezium connects to the source database and reads database transaction logs. For PostgreSQL, changes are captured from the WAL and published as CDC events.

The events contain information such as: operation type, before state, after state, primary key, source timestamp, transaction information, and source database metadata.

Typical operations include:

- `c` = create / insert
- `u` = update
- `d` = delete
- `r` = read / snapshot

The connector configuration uses a dedicated replication slot and publication. Example configuration:

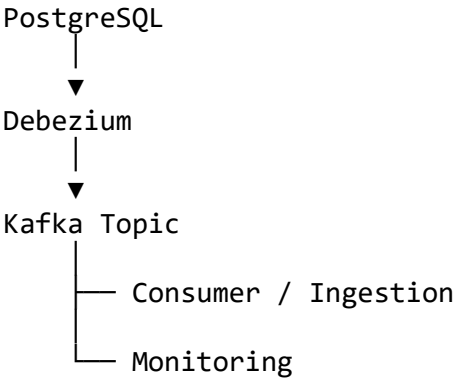
```
{
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "tasks.max": "1",
  "slot.name": "<dedicated_replication_slot>",
  "publication.name": "<cdc_publication>"
}
```

The replication slot ensures that PostgreSQL retains WAL information until Debezium has consumed it.

4. Kafka Layer

Debezium publishes CDC events into Kafka topics. Kafka provides durable event storage, decoupling between source and target, replay capability, horizontal scalability, fault tolerance, and consumer-based processing.

The general flow is:



Kafka topics are validated by checking: topic existence, partition count, latest offsets, consumer lag, message production, and message consumption.

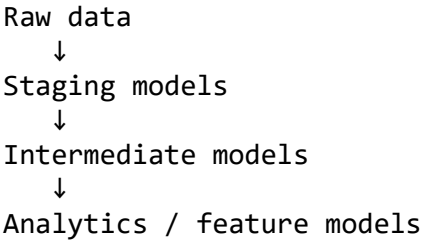
5. Apache Doris

Apache Doris is used as the analytical serving/storage layer. CDC events are ingested into Doris where they can be queried and transformed.

Doris provides analytical SQL, high-performance aggregation, streaming ingestion, table partitioning, materialized/analytical datasets, and operational monitoring. The target tables are structured to support analytical access and efficient incremental processing.

6. dbt Transformation Layer

dbt is used to transform the raw/staging data into analytics-ready datasets. The transformation layer separates:



dbt is also responsible for implementing SQL-based data quality checks. Examples include not-null validation, unique key validation, referential integrity, accepted values, row-count checks, and freshness checks.

Typical execution:

```
dbt deps
dbt debug
dbt run
dbt test
```

For production execution:

```
dbt build
```

`dbt build` combines model execution and associated tests.

7. Apache Airflow

Apache Airflow orchestrates the pipeline. The Airflow DAG manages the execution order and dependencies between pipeline components. Example:

```
Start
  │
  ▼
Validate Source
  │
  ▼
Validate CDC / Kafka
  │
  ▼
Validate Target Ingestion
  │
  ▼
Run dbt
  │
  ▼
Run Data Quality Tests
  │
  ▼
Update Monitoring Metrics
  │
  ▼
End
```

Airflow provides scheduling, retry handling, task dependencies, failure visibility, execution history, logs, and manual reruns.

8. End-to-End Setup

8.1 Prerequisites

Install the following: Git, Docker, Docker Compose, Python 3.x, PostgreSQL client, and Kafka client utilities where required.

Verify Docker:

```
docker --version
docker compose version
```

Verify Python:

```
python --version
```

9. Clone the Repository

```
git clone <repository-url>
cd senior-data-engineer-assessment
```

10. Environment Configuration

Create the environment file:

```
cp .env.example .env
```

Configure environment-specific values. Example:

```
POSTGRES_HOST=<host>
POSTGRES_PORT=<port>
POSTGRES_DB=<database>
POSTGRES_USER=<user>
POSTGRES_PASSWORD=<password>
KAFKA_BOOTSTRAP_SERVERS=<kafka-host>:9092
DORIS_HOST=<doris-host>
DORIS_PORT=<doris-port>
DORIS_DATABASE=<database>
DORIS_USER=<user>
DORIS_PASSWORD=<password>
```

Security

Credentials must not be committed to Git. Use environment variables, Docker secrets, CI/CD secrets, or cloud secret managers depending on the deployment environment.

11. Start the Platform

Start the platform using Docker Compose:

```
docker compose up -d
```

Check running services:

```
docker compose ps
```

Expected services include the pipeline/orchestration components used by the implementation. The implementation was validated with the Airflow container running successfully.

Figure: Docker Compose Services (insert screenshot — docs/images/01-docker-compose-ps.png)

12. Start / Validate Airflow

Access Airflow through:

```
http://localhost:8080
```

Log in using the credentials configured for the local environment. The Airflow UI provides visibility into DAGs, DAG runs, task states, logs, retries, execution duration, and scheduling.

Figure: Airflow DAG (insert screenshot — docs/images/02-airflow-dag.png)

Validate DAG

Confirm that: the DAG is visible, the DAG is unpaused, tasks are scheduled, tasks execute successfully, logs contain no errors, and failed tasks can be retried.

13. Validate the CDC Connector

The Debezium connector can be validated through the Kafka Connect REST API. Example:

```
curl http://localhost:8083/connectors
```

Check connector status:

```
curl http://localhost:8083/connectors/<connector-name>/status
```

A healthy connector should report:

```
connector.state = RUNNING
task.state      = RUNNING
```

Example connector configuration used by the project includes:

```
{
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "tasks.max": "1",
  "slot.name": "<replication-slot>",
  "publication.name": "<publication>"
}
```

Figure: Debezium / Kafka Connector (insert screenshot — docs/images/03-debezium-connector.png)

14. Validate Kafka

List topics:

```
kafka-topics.sh \
  --bootstrap-server <kafka-host>:9092 \
  --list
```

Describe a topic:

```
kafka-topics.sh \
  --bootstrap-server <kafka-host>:9092 \
  --describe \
  --topic <topic-name>
```

Consume events for validation:

```
kafka-console-consumer.sh \
  --bootstrap-server <kafka-host>:9092 \
  --topic <topic-name> \
  --from-beginning
```

Validation should confirm that CDC records are being produced. For example:

```
INSERT
   ↓
PostgreSQL
   ↓
Debezium
   ↓
Kafka
   ↓
CDC Event
```

An update to the source should result in a corresponding CDC event.

15. Validate Data Movement into Doris

Connect to Doris using the configured SQL client. Example:

```
mysql -h <doris-host> \
      -P <doris-port> \
      -u <doris-user> \
      -p
```

Check databases:

```
SHOW DATABASES;
```

Check tables:

```
SHOW TABLES FROM <database>;
```

Validate records:

```
SELECT *
FROM <database>.<table>
LIMIT 10;
```

Validate record counts:

```
SELECT COUNT(*)
FROM <database>.<table>;
```

Validate the latest ingestion:

```
SELECT MAX(<timestamp_column>)
FROM <database>.<table>;
```

16. Validate Source-to-Target Movement

A complete pipeline validation should not stop at checking that individual services are running. The following sequence should be performed:

Step 1 — Insert a test record. Insert a controlled test record into the source.

```
INSERT INTO <source_table> (...)
VALUES (...);
```

Step 2 — Confirm Debezium captured it. Check the connector status and Kafka topic.

Step 3 — Confirm Kafka received it. Consume the relevant topic.

Step 4 — Confirm Doris received it. Query the target table:

```
SELECT *
FROM <target_table>
WHERE <business_key> = '<test-key>';
```

Step 5 — Confirm transformation. Query the dbt model:

```sql
SELECT *
FROM <analytics_model>
WHERE <business_key> = '<test-key>';
```

Step 6 — Confirm monitoring. Check that: row count changed, latest ingestion timestamp changed, freshness status changed, and pipeline status remains healthy.

This provides true end-to-end validation.

17. Data Quality Validation

Data quality is validated at multiple layers.

Source validation — validate connectivity, source availability, schema availability, and expected tables.

CDC validation — validate connector status, replication slot, publication, Kafka topic, and consumer lag.

Target validation — validate record count, maximum ingestion timestamp, duplicate records, null values, and expected partitions.

Transformation validation — run:

```
dbt test
```

or:

```
dbt build
```

18. Freshness Monitoring

The monitoring implementation tracks whether tables are being refreshed according to their expected schedule. Supported scheduling patterns include daily, weekly, monthly, and on-demand.

The monitoring logic compares the latest available snapshot against the expected processing schedule. Example monitoring fields: `schema_name`, `table_name`, `snapshot_timestamp`, `last_process_date`, `row_count`.

The latest snapshot for each table is used to determine freshness. Example:

```sql
SELECT DISTINCT ON (table_name)
       table_name,
       snapshot_timestamp,
       last_process_date,
       row_count
FROM featurestore_monitoring_tables
ORDER BY table_name, snapshot_timestamp DESC;
```

19. Pipeline Health Score

The monitoring layer calculates a health score based on pipeline/data freshness indicators. The health score can incorporate table freshness, ingestion status, expected schedule, record availability, and processing success.

The final monitoring output provides an overall indication of whether the data platform is healthy. Example:

```
Overall Platform Health
        │
        ├── Freshness
        ├── Availability
        ├── Ingestion
        ├── Record Counts
        └── Pipeline Status
```

20. Grafana / Observability

PostgreSQL is used as a monitoring metadata store. Grafana can connect to the monitoring database and expose operational metrics. Example metrics include last ingestion time, pipeline lag, records per minute, table freshness, routine load status, record counts, and pipeline health.

Figure: Grafana Monitoring Dashboard (insert screenshot — docs/images/04-grafana-monitoring.png)

The monitoring database contains the metadata required to build operational dashboards without querying the source systems directly.

21. Qlik Monitoring Dashboard

Qlik can be used to provide business-facing monitoring views. The monitoring dashboard provides visibility into updated vs non-updated tables, table freshness, ingestion trends, record volumes, and time-based ingestion patterns.

Figure: Qlik Monitoring Dashboard (insert screenshot — docs/images/05-qlik-monitoring.png)

The dashboard uses the monitoring tables rather than placing additional workload on the transactional source.

22. Data Source Link and Authentication

The data source used by the pipeline is the configured source database used for CDC ingestion. Connection information is environment-specific and must be supplied through environment variables or deployment secrets. Example:

```
Host:       <SOURCE_HOST>
Port:       <SOURCE_PORT>
Database:   <SOURCE_DATABASE>
Username:   <SOURCE_USERNAME>
Password:   <SOURCE_PASSWORD>
```

Authentication is performed using credentials supplied through the secure runtime configuration.

Important

Credentials must never be stored directly in source code, DAG files, dbt models, connector JSON committed to Git, README files, or Docker images.

For the assessment repository, replace `<repository-url>`, `<SOURCE_HOST>`, `<SOURCE_PORT>`, and `<SOURCE_DATABASE>` with the appropriate non-secret endpoint information. Passwords and other secrets should remain in `.env`, Docker secrets, or CI/CD secret variables.

23. Database Access

PostgreSQL

Connect using:

```
psql \
  -h <host> \
  -p <port> \
  -U <username> \
  -d <database>
```

Apache Doris

Connect using:

```
mysql \
  -h <doris-host> \
  -P <doris-port> \
  -u <username> \
  -p
```

Kafka

Kafka is accessed using the configured bootstrap server:

```
<kafka-host>:9092
```

The exact host and credentials depend on the deployment environment.

24. Orchestrator Access

Airflow is the orchestration platform.

Local development:

```
http://localhost:8080
```

The Airflow interface provides DAG management, task execution, logs, retry controls, scheduling, historical runs, and failure investigation.

Figure: Airflow Pipeline Execution (insert screenshot — docs/images/06-airflow-run.png)

25. Platform Observability

The platform can be observed at several levels.

Component	What is monitored
PostgreSQL	Source availability and CDC
Debezium	Connector/task health
Kafka	Topics, offsets and consumer lag
Doris	Ingestion and table state
dbt	Model execution and tests
Airflow	DAG/task execution
PostgreSQL Monitoring DB	Data platform metrics
Grafana	Operational metrics
Qlik	Data freshness/business monitoring

This multi-layer approach ensures that an application can distinguish between:

```
Source problem
      ↓
CDC problem
      ↓
Kafka problem
      ↓
Ingestion problem
      ↓
Transformation problem
      ↓
Data quality problem
```

rather than only reporting that the final table is stale.

26. CI/CD

The project uses Git-based CI/CD to automate validation before changes are merged or deployed. The pipeline is triggered when changes are pushed to the repository or when a pull request is created. Typical trigger:

```
Developer
   │
   ▼
git push
   │
   ▼
CI/CD Pipeline
   │
   ├── Code validation
   ├── SQL/dbt validation
   ├── Unit/data tests
   ├── Configuration validation
   ├── Build
   └── Deployment
```

27. CI/CD Validation

The CI pipeline validates the following.

Python validation

Where applicable:

```
python -m compileall .
```

Linting and formatting checks can also be executed.

dbt validation

```
dbt deps
dbt parse
dbt compile
dbt test
```

For a full build:

```
dbt build
```

Docker validation

Build the application image:

```
docker compose build
```

Validate that services can start:

```
docker compose up -d
docker compose ps
```

SQL validation

SQL models are compiled and tested through dbt. This helps catch invalid SQL, missing relations, incorrect references, failed tests, and model dependency issues.

28. CI/CD Trigger Flow

```
Pull Request / Push
        |
        ▼
CI Pipeline
        |
        ├── Checkout source
        |
        ├── Install dependencies
        |
        ├── Validate Python
        |
        ├── Validate Docker
        |
        ├── dbt deps
        |
        ├── dbt parse / compile
        |
        ├── dbt tests
        |
        └── Build / deploy
```

A deployment should only proceed after the validation stages have completed successfully.

Figure: CI/CD Pipeline (insert screenshot — docs/images/07-cicd-pipeline.png)

29. Recommended Repository Structure

```
senior-data-engineer-assessment/
|
├── dags/
|   └── pipeline_dag.py
|
├── dbt/
|   ├── models/
|   ├── tests/
|   ├── macros/
```

```
│   └── dbt_project.yml
│
├── connectors/
│   └── debezium/
│       └── connector.json
│
├── sql/
│   ├── source/
│   ├── staging/
│   └── monitoring/
│
├── monitoring/
│   ├── grafana/
│   └── qlik/
│
├── tests/
│
├── docs/
│   └── images/
│       ├── 01-docker-compose-ps.png
│       ├── 02-airflow-dag.png
│       ├── 03-debezium-connector.png
│       ├── 04-grafana-monitoring.png
│       ├── 05-qlik-monitoring.png
│       ├── 06-airflow-run.png
│       └── 07-cicd-pipeline.png
│
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

30. End-to-End Validation Checklist

Before considering the pipeline successfully deployed, validate the following:

- ☐ Source database is reachable
- ☐ Source tables exist
- ☐ Debezium connector is running
- ☐ Replication slot is active
- ☐ Kafka topics exist
- ☐ Kafka receives CDC events
- ☐ Kafka consumer/ingestion is running
- ☐ Doris target tables exist

- ☐ Doris receives records
- ☐ Record counts are increasing as expected
- ☐ Latest ingestion timestamp is current
- ☐ dbt models compile successfully
- ☐ dbt transformations execute successfully
- ☐ dbt tests pass
- ☐ Airflow DAG is active
- ☐ Airflow tasks complete successfully
- ☐ Monitoring metrics are populated
- ☐ Freshness checks pass
- ☐ Grafana dashboards display metrics
- ☐ Qlik dashboards display freshness/monitoring information
- ☐ CI/CD validation passes

31. Troubleshooting

Airflow is not available

Check:

```
docker compose ps
```

Check logs:

```
docker compose logs airflow
```

Restart:

```
docker compose restart airflow
```

Debezium connector is not running

Check:

```
curl http://localhost:8083/connectors
```

Then:

```
curl http://localhost:8083/connectors/<connector-name>/status
```

Check Kafka Connect logs:

```
docker compose logs <kafka-connect-service>
```

Kafka receives no events

Validate that: PostgreSQL logical replication is enabled, the Debezium connector is running, the replication slot exists, the publication exists, the source table is included in the connector configuration, the Kafka topic exists, and Kafka Connect can reach Kafka.

Doris contains no records

Validate the entire chain:

```
Source
  ↓
Debezium
  ↓
Kafka
  ↓
Ingestion
  ↓
Doris
```

Do not troubleshoot Doris in isolation before confirming that Kafka is receiving events.

dbt model fails

Run:

```
dbt debug
```

Then:

```
dbt compile
```

and:

```
dbt test
```

Inspect the generated SQL and the failing model dependency.

32. Production Considerations

The implementation is designed with production engineering principles in mind.

Reliability — CDC rather than repeated full extraction, Kafka durability, Airflow retries, idempotent transformations, and failure isolation.

Scalability — Kafka partitions, distributed analytical storage, incremental transformations, and independent processing layers.

Observability — pipeline status, data freshness, ingestion lag, record counts, connector health, and data quality tests.

Security — credentials externalized, secrets excluded from Git, least-privilege access, and environment-specific configuration.

Maintainability — infrastructure separated from transformation logic, dbt models version controlled, Airflow DAGs version controlled, monitoring metadata centralized, and automated CI/CD validation.

33. Quick Start

For a new environment, the shortest execution path is:

```
# 1. Clone repository
git clone <repository-url>

# 2. Enter project
cd senior-data-engineer-assessment

# 3. Configure environment
cp .env.example .env

# 4. Start platform
docker compose up -d

# 5. Validate containers
docker compose ps

# 6. Validate Airflow
# Open http://localhost:8080

# 7. Validate CDC connector
curl http://localhost:8083/connectors

# 8. Validate Kafka
kafka-topics.sh \
  --bootstrap-server <kafka-host>:9092 \
  --list

# 9. Validate dbt
dbt debug
dbt deps
dbt build

# 10. Validate target data
#   SELECT COUNT(*) FROM <target_table>;

# 11. Validate freshness
#   SELECT MAX(<timestamp_column>) FROM <target_table>;
```

34. Evidence of Successful Implementation

The following screenshots provide evidence of the implemented pipeline and operational environment:

- **Docker / Platform Services** — Docker Compose Services
- **Airflow Orchestration** — Airflow DAG
- **CDC Connector** — Debezium Connector
- **Grafana Monitoring** — Grafana Monitoring
- **Qlik Monitoring** — Qlik Monitoring
- **Airflow Execution** — Airflow Execution
- **CI/CD** — CI/CD Pipeline

35. Conclusion

The solution demonstrates a complete production-oriented data engineering lifecycle:

```
SOURCE
  ↓
CDC
  ↓
DEBEZIUM
  ↓
KAFKA
  ↓
STREAMING INGESTION
  ↓
APACHE DORIS
  ↓
DBT
  ↓
ANALYTICS / FEATURE STORE
  ↓
DATA QUALITY
  ↓
MONITORING
  ↓
AIRFLOW ORCHESTRATION
  ↓
CI/CD
```

The architecture provides reliable data movement, scalable processing, automated transformation, observable execution, data quality validation, and controlled deployment. The result is an end-to-end platform that can support both analytical and machine-learning workloads while providing the operational visibility required for production environments.